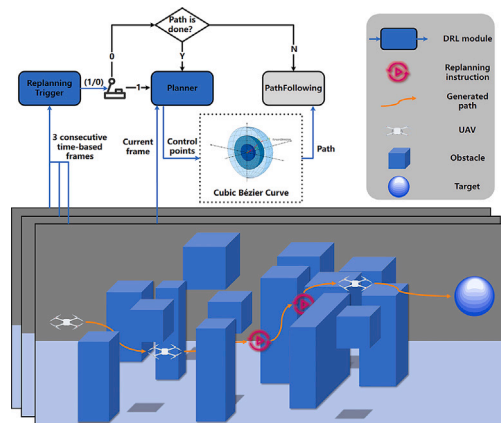




^d National Key Laboratory of Digital and Agile Aircraft Design, Chengdu, 610091, China

- A hierarchical method is proposed for UAVs' long-term planning and adaptive replanning.
- The action space using Bézier curves reduces the planning complexity.
- The adaptive replanning module enhances reliability in complex environments.
- The high-fidelity Unity 3D simulation platform ensures the models' stable convergence.

GRAPHICAL ABSTRACT



ARTICLE INFO

Keywords:

- Deep reinforcement learning
- Unmanned aerial vehicle
- Autonomous navigation
- Hierarchical adaptive planner

A B S T R A C T

Deep reinforcement learning (DRL) offers significant advantages for the autonomous navigation of unmanned aerial vehicles (UAVs), including being map-free, robust in uncertain environments, and having low requirements for sensor accuracy. Current DRL navigation methods primarily fall into two categories: reactive methods similar to local path planning, which lack long-term vision and suffer from local optima and path oscillation; and planning methods, which face challenges due to high-dimensional action spaces and unreliable path tracking. Based on the above problems, we propose a novel DRL-based Hierarchical Adaptive Navigation Planner (HAP), which enables the UAV to have both long-term vision and adaptive replanning capabilities. Specifically, our method involves two modules, namely Planner and Replanning Trigger. The former uses 3D Bézier control points to reduce the complexity of the path planning action space, while the latter adaptively triggers Planner for replanning. Furthermore, we design targeted dense reward functions for each module to ensure convergence, and use the latest Distributional Soft Actor-Critic (DSAC) algorithm to improve the Planner's performance ceiling. In the experimental part, we design challenging simulation environments of different complexities for UAV autonomous navigation tasks to verify the effectiveness of HAP and its advantages over other methods. Experimental results in various scenarios show that under the same input, the comprehensive performance of HAP is better than other DRL-based reactive navigation methods, and it can achieve an adaptive balance between real-time obstacle avoidance and long-term planning.

* Corresponding author at: College of Computer Science, Sichuan University, Chengdu, 610065, China.

Email address: guanxiongli@scu.edu.cn (G. Li).

1. Introduction

With its high mobility, low risk and significantly reduced labor cost, the unmanned aerial vehicle (UAV) has shown broad application prospects in both military and civilian fields [1–3]. Although UAVs perform diverse tasks, the prerequisite for their operational success is to safely and effectively avoid obstacles and reach the target location. Therefore, how to ensure the safe and efficient autonomous navigation of UAVs has become an increasingly important research focus [4]. Traditional methods rely on a series of non-learning-based algorithms to address UAV autonomous navigation. This kind of approach typically requires highly accurate global maps and involves a sequential process encompassing localization, mapping, optimal path selection, motion planning, and control [5,6], which leads to high costs and reduced efficiency [7].

The rapid development of artificial intelligence has promoted the widespread application of learning-based methods in solving the autonomous navigation of UAVs [8]. Among these methods, deep reinforcement learning (DRL)—built on reinforcement learning (RL) and deep neural networks (DNN)—stands out as one of the most competitive options, thanks to its map-free, strong capability to handle highly dynamic and uncertain environments, and low requirements for sensor accuracy [9]. Most existing DRL-based navigation methods can be divided into two categories: reactive and planning-based. However, they both encounter challenges in increasingly complex task scenarios. Reactive methods usually lack long-term planning capability: they tend to fall into local optimality in complex environments, get trapped due to robot stagnation, and their overly frequent action outputs may also cause oscillations in the navigation path. In contrast, planning-based methods often face issues such as excessively high action space dimensions and insufficient reliability of the output path in the later stages of tracking.

As an effort to address the above-mentioned issues, we propose a novel DRL-based method named Hierarchical Adaptive Navigation Planner (HAP), which aims to enable the UAV to output paths for tracking in a more complex and dynamic three-dimensional environment, while having the ability to decide when to replan to deal with dynamic threats in the environment. The principal contributions of this paper are enumerated below.

- I. A DRL-based hierarchical navigation framework including two modules named Planner and Replanning trigger is proposed to enable the UAV to have both long-term planning and adaptive replanning capabilities.
- II. The Planner module based on 3D Bézier control point is designed to reduce the complexity of the action space used for path output, while the Distributional Soft Actor-Critic algorithm (DSAC) [10] is used to train Planner to more stably approach the optimal strategy.
- III. The DQN-based Replanning trigger is designed to adaptively issue replanning instructions to Planner, enabling the UAV to demonstrate higher generalization in more complex and highly dynamic challenging environments.
- IV. Unity 3D is used to build a high-fidelity simulation environment, and the dense reward function is well designed to ensure the convergence, comparison, and effectiveness verification of algorithms.

The remainder of this paper is structured as follows: Section 2 introduces some related works. Section 3 introduces our proposed method HAP based on the modeling of the autonomous navigation task. In Section 4, we talk about the experiment setup and simulation results to verify the effectiveness of HAP. Lastly, we end the article with the conclusion in Section 5.

2. Related work

Based on the Markov Decision Process (MDP), RL is a machine learning method that makes decisions and learns based on a reward feedback

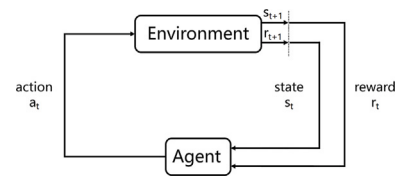


Fig. 1. The process of reinforcement learning.

mechanism. As seen in Fig. 1, in the process of RL, the agent (an entity capable of executing actions) iteratively interacts with its environment and progressively refines its decision-making policy by responding to reward signals provided by the environment.

Since Deep Q-Network (DQN) was first proposed and applied to video games [11], DRL has played a significant role in many fields, such as robotics [12], transportation [13], and competition [14]. In terms of autonomous navigation, most of the early research focused on using DQN or its variants to guide the autonomous navigation of UAVs [15–17]. However, due to the inherent algorithmic limitations of DQN, these approaches are restricted to discrete action spaces. Consequently, UAV maneuvers can only be modeled via discrete translations or rotational adjustments, leading to mismatches between simulation and reality. This constraint presents substantial challenges for adaptation in complex environments.

As Actor-Critic (AC) based algorithms are proposed, researchers begin to focus on more complex continuous control problems. Wang et al. [18] designed a prior control policy called non-expert helper to help the DRL model learn appropriate navigation strategies in environments with sparse rewards. Liu et al. [19] used a series of optimization measures such as an averaged estimate function, recurrent mechanism, temporal attention, and a hierarchical framework to enable UAVs to better perform navigation actions in continuous action space. Although these works have advanced continuous control for UAV navigation, they are largely limited to simplified environments: either 2D scenarios with low-dimensional state-action spaces, or 3D environments enforcing fixed altitudes. This limitation results in a significant gap between simulation and real-world applicability.

Researchers have increasingly focused on UAV navigation within complex, dynamic 3D environments. Zhang et al. [20] proposed a two-stream network structure to help UAVs perform navigation tasks in 3D, dynamic, multi-obstacle environments. Li et al. [21] developed a reinforcement learning framework based on human-in-the-loop for UAVs in a high-fidelity 3D environment taking into account altitude changes, thereby improving navigation success and efficiency. Zhang et al. [22] designed an efficient state-decomposition Deep Deterministic Policy Gradient (DDPG) algorithm, which effectively improved the navigation performance of UAVs in 3D complex environments.

More recently, significant efforts have been devoted to developing hybrid architectures and advanced decision-making strategies to tackle the uncertainties in complex 3D environments. To address the limitations of standard end-to-end learning, researchers have begun integrating DRL with global planners or refining state representations. Xue et al. [23] modeled the navigation of UAVs in complex 3D environments as a Partially Observable Markov Decision Process (POMDP) and derived a DRL algorithm based on the stochastic value function to solve it. Subsequently, they refined this approach by developing hybrid navigation frameworks that integrate DRL with motion planners [24,25] and employ event-triggered strategies [26] for robust navigation in more complex, unknown scenarios. Recently, Luo et al. [27] proposed a DRL algorithm using a novel tangent path reward to enable quadrotor navigation while explicitly modeling attitude changes.

Despite the advancements in robustness, similar to reactive local path planning, many of these methods still encounter challenges such

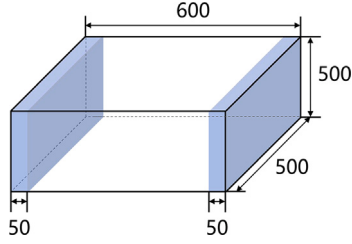


Fig. 2. The environment modeling for autonomous navigation.

as local optima [28,29], the freezing robot problem [30,31], and path oscillations in complex environments. To enable DRL models to achieve long-term planning without significantly increasing action space complexity, Yu et al. [32] proposed a parameterization method using Bézier curve control points for 2D mobile robot navigation. However, this approach is limited to 2D environments. Moreover, replanning only upon path completion critically compromises reliability due to delayed responses to sudden dynamic threats.

Compared with the above methods, HAP synthesizes the complementary strengths of reactive and planning-based navigation paradigms. Specifically, we extend the path planning to complex 3D environments. Concurrently, we introduce an adaptive replanning module to enhance responsiveness in complex dynamic scenarios. This integrated framework enables UAVs to dynamically balance real-time obstacle avoidance with long-term planning.

3. Method

3.1. Problem formulation

3.1.1. Scenario description

As depicted in Fig. 2, the task scenario is modeled as a 600 m × 500 m × 500 m cuboid. Start and target points are randomly generated within the blue areas at opposite ends, while obstacles are randomly distributed throughout the remaining area. Notably, obstacles may be airborne to enhance environmental complexity and test UAVs' navigation capabilities in 3D space. In autonomous navigation tasks, UAVs should reach the target from the start point as quickly as possible while avoiding collisions. This objective can be expressed as:

$$\begin{cases} \min \sum_{t=0}^T \|\mathbf{p}_{t+1} - \mathbf{p}_t\|_2 \\ s.t. \quad o_{min} > \bar{o} \\ \mathbf{p}_0 = [x_{start}, y_{start}, z_{start}], \mathbf{p}_T = [x_{target}, y_{target}, z_{target}] \end{cases} \quad (1)$$

where T represents the total time of an episode of the task, $\bar{o}$ is the safety distance threshold between the UAV and obstacles, and $\mathbf{p}_0$ and $\mathbf{p}_T$ represent the positions of the starting point and the target point in the environment.

As for the environmental information that the UAV can perceive, we express it as:

$$S_{UAV} = [\text{selfInfo}, \text{obsInfo}] \quad (2)$$

As shown in Fig. 3, selfInfo is the information about the UAV itself measured by the Inertial Measurement Unit (IMU) and the Global Positioning System (GPS), which can be expressed as:

$$\text{selfInfo} = [x, y, z, x_{target} - x, y_{target} - y, z_{target} - z, \omega, \mu] \quad (3)$$

where (x, y, z) represents the absolute position of the UAV, ω is the angle between the UAV's forward direction in the XOY plane and the X-axis, and μ is the angle between the forward direction vector and the

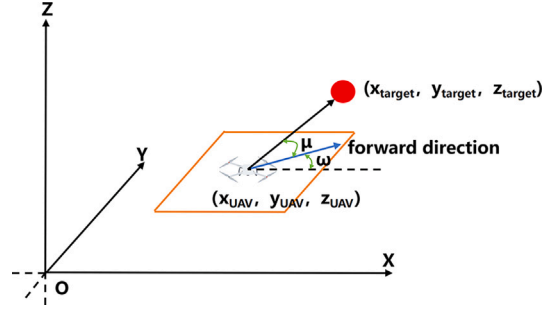


Fig. 3. The environment modeling for autonomous navigation.

vector pointing from its own position to the target. Furthermore, we incorporate the relative target position $(x_{target} - x, y_{target} - y, z_{target} - z)$ into the state representation. This augmentation enhances target-oriented guidance and accelerates training convergence.

Typically, UAVs rely on onboard sensors (cameras or laser rangefinders) for environmental perception in autonomous navigation [33]. To mitigate generalization issues caused by varying lighting conditions and diverse surface properties, we employ dense LiDAR for obstacle distance measurement. Specifically, 18 LiDAR planes are arranged around the UAV at $\pi/36$ rad intervals along the X-axis. Each plane emits rays up to INF_R length at $\pi/36$ rad angular resolution, forming a full-coverage 648-beam sensing system. Fig. 4 shows the ray distribution of the design in the XOY plane and the XOZ plane.

Under this design, we express the obstacle ranging information obsInfo as:

$$\begin{aligned} \text{obsInfo} &= [o_{1,1}, o_{1,2}, \dots, o_{1,36}, \dots, o_{2,1}, o_{2,2}, \dots, o_{i,j}], \\ o_{i,j} &= \begin{cases} \text{INF_R} & \text{if no obstacles detected} \\ \text{detected_distance} & \text{otherwise} \end{cases}, \quad i \in [1, 18], j \in [1, 36] \end{aligned} \quad (4)$$

where $o_{i,j}$ denotes the detection data of the j -th beam on the i -th LiDAR plane.

3.1.2. Kinematics of the UAV

The most commonly used quad-rotor UAVs utilize four motors to regulate rotor speeds, enabling attitude control and thrust adjustment through mature low-level controllers. Consequently, we define the action space as translational movements along forward, lateral, and vertical axes, with steering governed by attitude changes. The executable actions of the UAV can be expressed as:

$$\begin{aligned} A_{UAV} &= [a_{Forward}, a_{Right}, a_{Up}, a_{Rotation}], \\ a_{Forward}, a_{Right}, a_{Up}, a_{Rotation} &\in [-1, 1] \end{aligned} \quad (5)$$

As illustrated in Fig. 5, UAV translation along the body-fixed X_{UAV} axis (forward/backward) is controlled by $\text{MAX_V} \cdot a_{Forward}$, while motion along Y_{UAV} (lateral) is governed by $\text{MAX_V} \cdot a_{Right}$. Vertical movement along Z_{UAV} is executed via $\text{MAX_V} \cdot a_{Up}$, where MAX_V denotes an environmental hyperparameter defining maximum maneuver velocity. Rotational motion about Z_{UAV} is regulated by $0.1 \cdot a_{Rotation}$, modifying the heading direction. The body-fixed coordinate system $(X_{UAV}, Y_{UAV}, Z_{UAV})$ aligns X_{UAV} with the forward direction and maintains Z_{UAV} orthogonal to the XOY plane.

Moreover, to ensure UAV motion complies with physical constraints while preserving control intent, we implement a velocity scaling mechanism. This approach maintains directional intent of control inputs while enforcing the MAX_V velocity limit. Consequently, UAV kinematics at each time step can be formulated as:

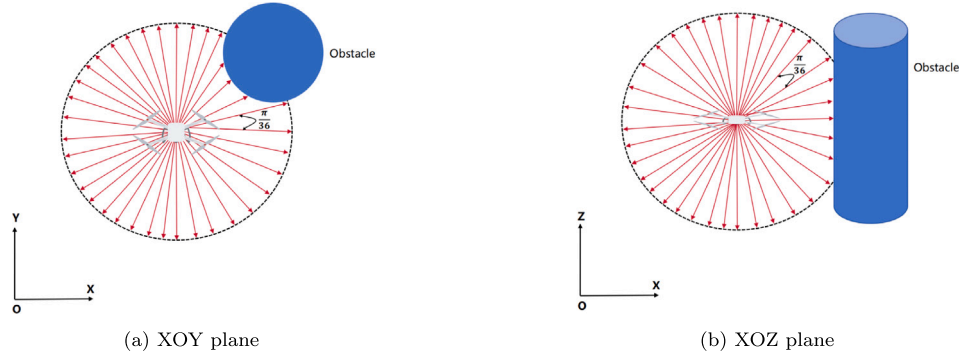


Fig. 4. The diagram of obstacle distance measurement.

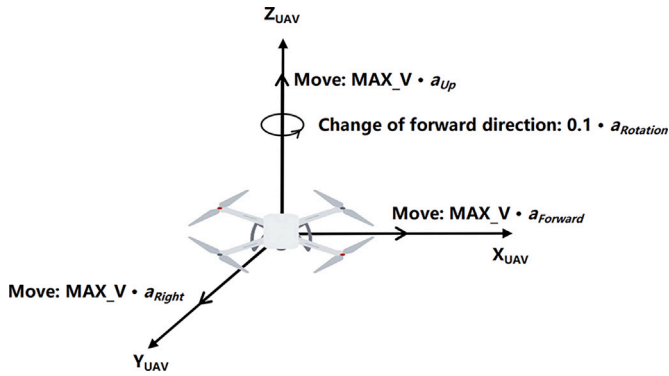


Fig. 5. The executable actions of the UAV.

$$\begin{cases} \omega_{t+1} = (\omega_t - 0.1 \cdot a_{\text{rotation}}) \bmod 2\pi \\ x_{t+1} = x_t + \text{TIME_STEP} \cdot \text{MAX_V} \cdot \kappa \cdot (a_{\text{forward}} \cdot \cos \omega_{t+1} - a_{\text{right}} \cdot \sin \omega_{t+1}) \\ y_{t+1} = y_t + \text{TIME_STEP} \cdot \text{MAX_V} \cdot \kappa \cdot (a_{\text{forward}} \cdot \sin \omega_{t+1} + a_{\text{right}} \cdot \cos \omega_{t+1}) \\ z_{t+1} = z_t + \text{TIME_STEP} \cdot \text{MAX_V} \cdot \kappa \cdot a_{\text{up}} \end{cases}$$

$$\text{where } \kappa = \min \left(1, \frac{\text{MAX_V}}{\|\mathbf{v}'\|} \right), \mathbf{v}' = \begin{bmatrix} a_{\text{forward}} \cdot \cos \omega_{t+1} - a_{\text{right}} \cdot \sin \omega_{t+1} \\ a_{\text{forward}} \cdot \sin \omega_{t+1} + a_{\text{right}} \cdot \cos \omega_{t+1} \\ a_{\text{up}} \end{bmatrix} \quad (6)$$

It is worth noting that both the planning-based and reactive methods utilize the executable actions mentioned above to control the UAV. The key distinction is that the planning-based method first outputs a path and then uses these actions to track it, whereas reactive methods directly output these actions in real time.

3.2. Hierarchical adaptive navigation planner

To equip the UAV with both long-term planning and real-time obstacle avoidance capabilities, we propose a novel hierarchical navigation framework called Hierarchical Adaptive Navigation Planner (HAP). As seen in Fig. 6, HAP comprises two DRL modules, namely Planner and Replanning Trigger. Planner is responsible for observing the current environment and outputting discrete Bézier curve control points, aiming to generate a collision-free long-term path that approaches the target point. Replanning Trigger monitors the environment in real time to determine whether an immediate call to Planner for path update is necessary.

3.2.1. Planner

The role of Planner is to observe the environment and output a smooth, collision-free path that approaches the target point. We model this process as an MDP, which is defined by a tuple $\{S, A, R, P, \gamma\}$, where S is the state space, A is the action space, R is the reward function, P is the state transition probability and $\gamma \in [0, 1]$ is the discount factor.

1) *State space*: In our formulation, the state space of Planner is equivalent to the information in Eq. (2), that is:

$$S^{\text{Planner}} = [\text{selfInfo}, \text{obsInfo}] \quad (7)$$

2) *Action space*: To enable Planner to output a future path over a period without dramatically increasing the dimensionality of its action space, we have Planner output n control points and a Bézier curve is then generated as the path based on the UAV's current position. In this paper, we set $n = 3$ to ensure Planner can generate 3D Bézier curves while reducing the action space dimensionality. The action space of Planner can be expressed as:

$$A^{\text{Planner}} = [\theta^1, \phi^1, \theta^2, \phi^2, \theta^3, \phi^3] \quad (8)$$

where θ^n and ϕ^n represent the polar angle and azimuth angle, respectively, which are used to determine the direction of the n -th control point in the spherical coordinate system.

As shown in Fig. 7, at time t , the action output by Planner uniquely determines the three points P_t^1 , P_t^2 and P_t^3 on three hemispheres with equal radial differences in the forward direction (along the X_{UAV} axis) of the UAV. We use $\frac{\text{INF_R}}{3}$ as the radial difference of these three hemispheres, which aligns with intuition, as the reliability of paths beyond the detection range typically decreases sharply. In this setting, the UAV can plan a path within the detection range in its body coordinate system:

$$\begin{aligned} \bar{p}_t &= \text{Bézier}(O_t, P_t^1, P_t^2, P_t^3), \\ O_t &= (0, 0, 0), \\ P_t^n &= \frac{n \cdot \text{INF_R}}{3} (\sin \theta_t^n \cos \phi_t^n, \sin \theta_t^n \sin \phi_t^n, \cos \theta_t^n), n = 1, 2, 3 \end{aligned} \quad (9)$$

where $\bar{p}_t$ denotes the path generated at time t , O_t is the origin of the UAV's body coordinate system (i.e., the UAV's current position at time t) and INF_R represents the maximum detection range of the UAV's sensors.

3) *Reward function*: The DRL policy learns the required behavior by maximizing the cumulative reward. In HAP, the goal of Planner is to generate a collision-free path as close to the target as possible, while also being sufficiently smooth to reduce the UAV's tracking difficulty. Therefore, we define the reward function of Planner as follows:

$$R^{\text{Planner}} = r_{\text{target}} + r_{\text{safe}} + r_{\text{time}} + r_{\text{curvature}} + r_{\text{shaping}} + r_{\text{avoidance}} \quad (10)$$

where r_{target} , r_{safe} and r_{time} are sparse rewards that are only activated when the UAV triggers a key event, while $r_{\text{curvature}}$, r_{shaping} and $r_{\text{avoidance}}$

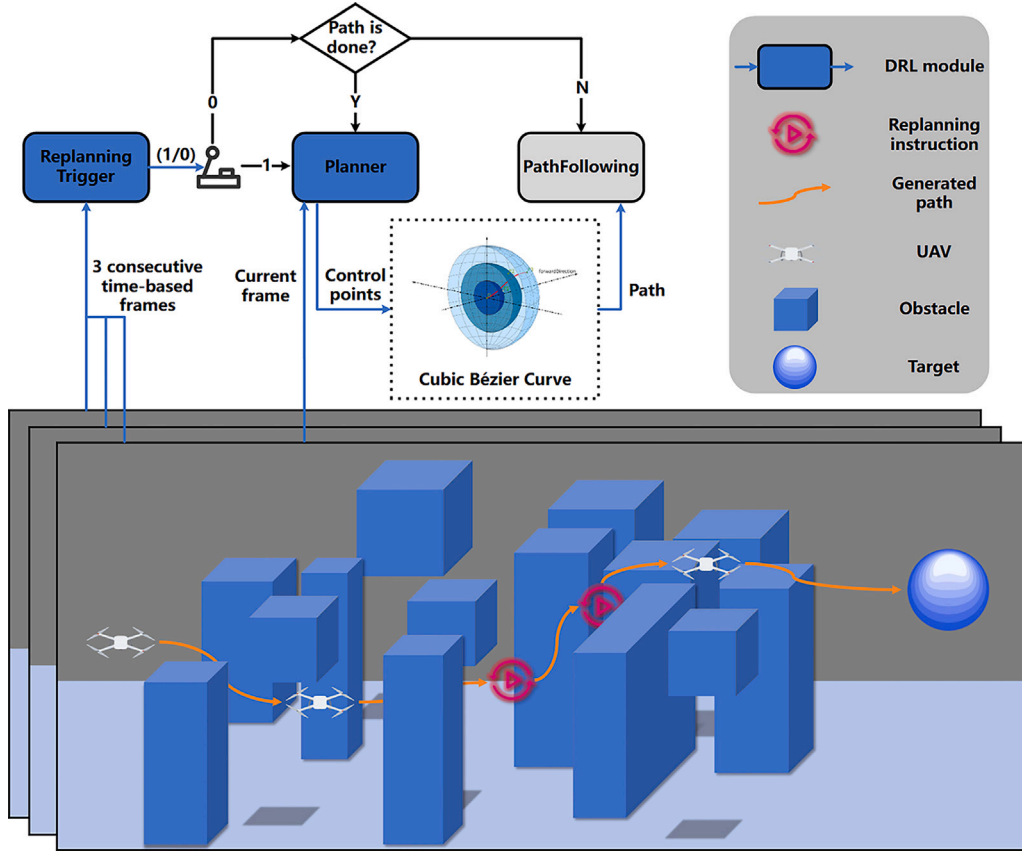


Fig. 6. The framework of HAP.

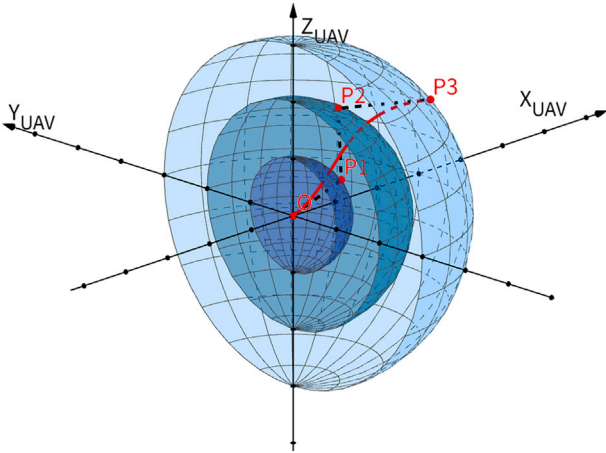


Fig. 7. The diagram of the path generation.

are dense rewards that provide timely feedback to Planner at each planning step, which helps ensure the convergence of the DRL algorithm.

Specifically, when the UAV reaches the target, Planner obtains a large reward:

$$r_t^{\text{target}} = \begin{cases} r_{\text{arrive}}, & \text{if the target is reached} \\ 0, & \text{otherwise} \end{cases} \quad (11)$$

where r_{arrive} denotes a configurable positive hyperparameter, representing the reward awarded when the UAV reaches the target.

Planner incurs a larger penalty when the UAV collides with an obstacle or flies out of environment bounds:

$$r_t^{\text{safe}} = \begin{cases} r_{\text{crash}}, & \text{if collision or out of bounds} \\ 0, & \text{otherwise} \end{cases} \quad (12)$$

where r_{crash} is a configurable negative hyperparameter.

If the UAV remains in the task environment for too long, Planner incurs a large penalty:

$$r_t^{\text{time}} = \begin{cases} r_{\text{timeout}}, & \text{if } t == \text{MAX_STEP} \\ 0, & \text{otherwise} \end{cases} \quad (13)$$

where MAX_STEP is a hyperparameter of the environment that defines the maximum number of time steps per episode of the UAV's task. r_t^{time} denotes a configurable negative hyperparameter that determines the specific value of the penalty.

In order to enable Planner to output smoother paths, we limit the curvature of the generated path using $r^{\text{curvature}}$:

$$r_t^{\text{curvature}} = \frac{\sigma_1}{N} \sum_{i=0}^{N-1} k(i), \quad (14)$$

$$k(i) = \frac{|\mathbf{B}'(\mathbf{p}_t^i) \times \mathbf{B}''(\mathbf{p}_t^i)|}{|\mathbf{B}'(\mathbf{p}_t^i)|^3}$$

where N denotes the number of sampling points along the path $\tilde{p}_t$, $k(i)$ represents the curvature of the path at the i -th sampling point $\mathbf{p}_t^i$, $\mathbf{B}'$ and $\mathbf{B}''$ are the first and second derivatives of the cubic Bézier curve

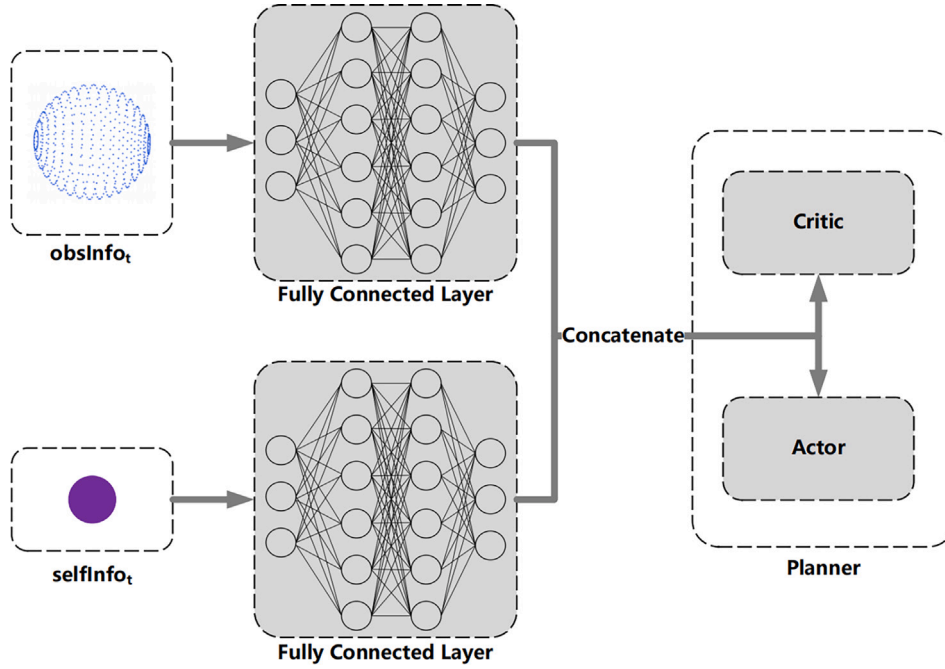


Fig. 8. The network architecture of Planner.

function respectively, and σ_1 is a negative hyperparameter used to adjust the reward magnitude.

As for r^{shaping} , it draws inspiration from artificial potential fields, where a constructed potential function guides Planner to generate paths that remain proximal to the target:

$$r_t^{\text{shaping}} = \sigma_2 (\|\mathbf{p}_t^0 - \mathbf{p}_{\text{target}}\| - \|\mathbf{p}_t^{N-1} - \mathbf{p}_{\text{target}}\|) \quad (15)$$

where σ_2 is a positive hyperparameter that adjusts the reward magnitude. The intuition behind r^{shaping} is to encourage Planner to generate a path as close to the target as possible in each planning step.

Finally, we use $r^{\text{avoidance}}$ to prevent the path generated by Planner from being excessively close to obstacles:

$$r_t^{\text{avoidance}} = \begin{cases} \sigma_3 e^{\sigma_4 \min d_i(i)}, & \text{if } \min d_i(i) < \frac{\text{INF_R}}{5}, \\ 0, & \text{otherwise} \end{cases}, \quad i = 0, 1 \dots N-1 \quad (16)$$

where σ_3 and σ_4 are negative hyperparameters that adjust the reward magnitude, and $d_i(i)$ denotes the distance from the i -th path point $\mathbf{p}_i^i$ to the closest obstacle within the INF_R range.

4) *Learning algorithm*: The output of Planner essentially involves selecting three control points on three hemispheres with equal radial spacing. This constitutes a relatively large action range, requiring it to explore as extensively as possible to find the optimal choice under given observations. Among existing algorithms, the Soft Actor-Critic algorithm (SAC) can adaptively explore the action space by modeling action outputs as a Gaussian action probability distribution. However, its Critic estimates only the expected Q-value. In our complex scenarios with randomized map topologies, the return distribution can be highly multimodal and skewed. Relying solely on the expectation may lead to Q-value overestimation and unstable convergence. Therefore, we adopt DSAC to model the full probability distribution of Q-values.

Specifically, distinct from standard SAC which directly approximates the expected $Q(s, a)$, DSAC [10] models the Q-function as a continuous probability distribution $Z(s, a)$. This value distribution is modeled as a Gaussian distribution $\mathcal{N}(\mu_\theta(s, a), \sigma_\theta^2(s, a))$, where μ_θ and σ_θ represent the mean and standard deviation estimated by the critic network.

The learning objective is to minimize the Kullback-Leibler (KL) divergence between the current value distribution and the target distribution defined by the distributional Bellman operator:

$$\mathcal{T}_D^\pi Z(s, a) \stackrel{D}{=} r(s, a) + \gamma (Z(s', a') - \alpha \log \pi(a' | s')) \quad (17)$$

where $Z(s', a')$ follows the target distribution, and α is the temperature parameter. By explicitly optimizing the variance σ_θ^2 alongside the mean, the algorithm effectively captures the aleatoric uncertainty arising from the randomized map topologies, preventing the policy from overfitting to high-variance transitions. This allows the agent to capture the uncertainty inherent in value estimation across diverse layouts, thereby improving the robustness and performance of Planner.

3.2.2. Replanning trigger

Based on Planner, we design another DRL-based module, Replanning Trigger, which enables the UAV to perform adaptive replanning. Specifically, Planner replans a new path not only when it completes tracking the current path $\bar{p}_t$, but also when it receives a trigger signal from Replanning Trigger. Similar to the modeling of Planner, we also model the operation of Replanning Trigger as an MDP.

- 1) State space: Unlike Planner, which relies on information at the current moment to make decisions, Replanning Trigger requires additional dynamic environmental information to make appropriate judgments and ensure the rationality of replanning instructions. Therefore, we define the state of Replanning Trigger at time t as follows:

$$s_t^{\text{Trigger}} = [\text{selfInfo}_{t-2:t}, \text{obsInfo}_{t-2:t}, \text{pp}_t], \quad \text{pp}_t \in [0, 1) \quad (18)$$

where the inclusion of three consecutive time steps of information (i.e., $\text{selfInfo}_{t-2:t}$ and $\text{obsInfo}_{t-2:t}$) allows Replanning Trigger to capture dynamic environmental changes and generate appropriate replanning instructions. Meanwhile, the path progress pp_t ensures awareness of the current path tracking status, helping to avoid excessive replanning frequency.

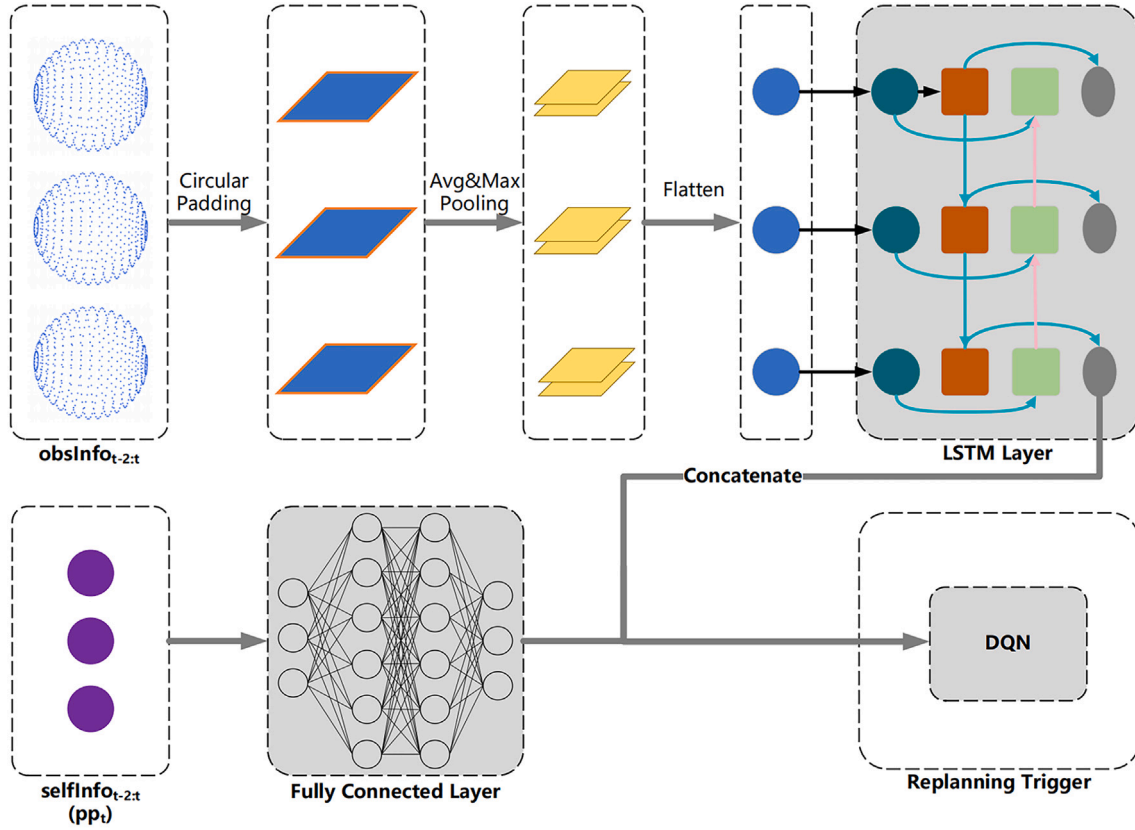


Fig. 9. The network architecture of Replanning Trigger.

- 2) Action space: Since the core decision of Replanning Trigger is a binary decision of whether to invoke Planner, its action space is designed as a discrete binary set:

$$a_t^{\text{Trigger}} = [\text{IsReplan}_t], \quad \text{IsReplan}_t \in \{0, 1\} \quad (19)$$

where IsReplan_t is a binary value that determines whether Planner needs to be triggered for replanning at time t .

- 3) Reward function: Replanning Trigger is required to issue replanning instructions to Planner in a timely manner to cope with complex and dynamic obstacle environments, so the reward function we design for it can be expressed as:

$$R^{\text{Trigger}} = r^{\text{target}} + r^{\text{safe}} + r^{\text{time}} + r^{\text{progress}} + r^{\text{replan}} \quad (20)$$

where r^{target} , r^{safe} and r^{time} are identical to their counterparts in Planner's reward function. r^{progress} and r^{replan} are dense reward functions that provide timely feedback to Replanning Trigger at each time step t .

To avoid wasting computing resources and generating excessively oscillatory flight trajectories due to overly frequent replanning, we use r^{progress} to further guide Replanning Trigger to issue replanning instructions only at appropriate time steps:

$$r_t^{\text{progress}} = \begin{cases} \sigma_5 \text{pp}_t, & \text{if } a_t^{\text{Trigger}} = 1 \\ 0, & \text{otherwise} \end{cases} \quad (21)$$

where σ_5 is a negative hyperparameter that adjusts the reward magnitude. The intuition behind r^{progress} is to impose a certain cost on Replanning Trigger each time it issues a replanning instruction, which is acceptable when a major risk (collision or out of bounds) is imminent.

Simply suppressing replanning may prevent the model from responding to complex task scenarios in a timely manner. Thus, we design r^{replan} to evaluate the effectiveness of each replanning instruction:

$$r_t^{\text{replan}} = \begin{cases} \sigma_6 \left(\min_{i=0}^{N-1-I} d(\vec{p}_{\text{new},i}) - \min_{j=I}^{N-1} d(\vec{p}_{\text{old},j}) \right), & \text{if } a_t^{\text{Trigger}} = 1 \\ 0, & \text{otherwise} \end{cases} \quad (22)$$

where I is the index of the path point before replanning. $d(\vec{p}_{\text{old},i})$ and $d(\vec{p}_{\text{new},j})$ represent the distances to the nearest detected obstacles at corresponding path points before and after replanning, respectively. That is, if the UAV can avoid obstacles more easily after replanning its path than before, we encourage such replanning behavior.

- 4) Learning algorithm: For training Replanning Trigger, a DRL algorithm capable of handling binary actions suffices. Our implementation employs DQN, where a neural network-based Q-function approximator learns optimal policies by maximizing expected returns. Furthermore, to mitigate training instability, we incorporate Double Q-learning and target network techniques.

3.3. Neural network design

Planner only needs to focus on current frame information. LiDAR ranging information and other types of data are encoded separately via a fully connected layer. The network structure of Planner is shown in Fig. 8.

The state space of Replanning Trigger encompasses three consecutive time-based frames, including $\text{selfInfo}_{t-2:t}$, $\text{obsInfo}_{t-2:t}$ and pp_t . We process the ranging information of the LiDAR separately from other information. Specifically, $\text{obsInfo}_{t-2:t}$ is expanded into a 18×36 two-dimensional vector, and then downsampled through max pooling and

Algorithm 1 HAP for the generation of paths.

```

1: Initialize the policy network  $\pi^{\text{Planner}}$ , the value network  $Q^{\text{Planner}}$ ,
    $Q^{\text{Trigger}}$ , experience buffer  $D^{\text{Planner}}$ ,  $D^{\text{Trigger}}$  and various hyperparam-
   eters.
2: for episode = 1 to  $E$  do
3:   for step  $t = 1$  to  $T$  do
4:      $a_t^{\text{Planner}} = \pi^{\text{Planner}}(s_t^{\text{Planner}})$ .
5:      $\tilde{p}_t = \text{Bezier}(a_t^{\text{Planner}})$ .
6:      $s_{t+1}^{\text{Planner}}, r_t^{\text{Planner}} = \text{pathFollowing}(\tilde{p}_t)$  till done.
7:      $D^{\text{Planner}} \leftarrow D^{\text{Planner}} \cup \{s_t^{\text{Planner}}, a_t^{\text{Planner}}, r_t^{\text{Planner}}, s_{t+1}^{\text{Planner}}\}$ .
8:     Sample batch  $B \sim D^{\text{Planner}}$  to update Planner by DSAC.
9:   end for
10: end for
11: Fix the trained  $\pi^{\text{Planner}}$ .
12: for episode = 1 to  $E$  do
13:   set  $\text{pp}_1 = 1$ .
14:   for step  $t = 1$  to  $T$  do
15:      $a_t^{\text{Trigger}} = \arg \max_{a^{\text{Trigger}}} Q(s_t^{\text{Trigger}}, a^{\text{Trigger}})$ .
16:     if  $a_t^{\text{Trigger}} == 1$  or  $\text{pp}_t == 1$  then
17:        $a_t^{\text{Planner}} = \pi^{\text{Planner}}(s_t^{\text{Planner}})$ .
18:        $\tilde{p}_t = \text{Bezier}(a_t^{\text{Planner}})$ .
19:     end if
20:      $s_{t+1}^{\text{Trigger}}, r_t^{\text{Trigger}} = \text{pathFollowing}(\tilde{p}_t)$  for one step.
21:     Sample batch  $B \sim D^{\text{Trigger}}$  to update Replanning Trigger by
       DQN.
22:   end for
23: end for

```

average pooling respectively. Finally, this information is passed through an LSTM layer to extract temporal information. Therefore, the network structure of Replanning Trigger is shown in Fig. 9.

It is challenging to train both Planner and Replanning Trigger simultaneously in a 3D complex task environment, as it is difficult to deal with the non-stationarity caused by simultaneously changing policies [34]. To address this challenge, we introduce a staged bottom-to-top training approach. Planner undergoes pre-training to master path planning, after which its parameters are frozen to serve as a fixed path generator. Replanning Trigger is then trained using this fixed Planner within the same task environment. The training process of HAP is shown in Algorithm 1.

4. Experiments

In this section, we introduce the simulation environment and present the training and testing results of various algorithms to validate the efficacy, adaptability, and generalization capability of HAP under diverse conditions.

4.1. Experiment platform and settings

To simulate 3D navigation task environments with complex, dense, and dynamic obstacles, we build a UAV navigation simulation platform based on the Unity ML-Agents framework. This platform not only delivers high-fidelity navigation tasks but also provides reliable environmental support for the training of DRL algorithms and the evaluation of their performance. A typical visualization of this platform is shown in Fig. 10.

The specific parameter settings of the simulation environment are shown in Table 1. Some of these parameters are configurable and closely tied to obstacle characteristics—allowing flexible simulation of different environmental complexities. Other parameters remain fixed; this is primarily to ensure the consistency of basic environmental conditions, which in turn facilitates the control of variables for analysis and guarantees the validity of experimental results.

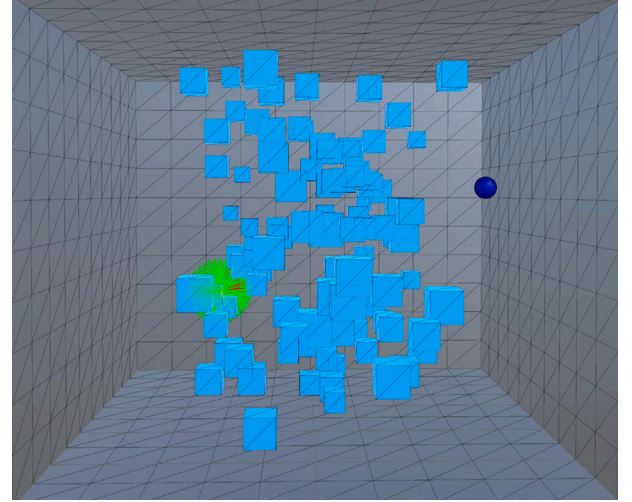


Fig. 10. The typical visualization of the three-dimensional simulation platform. The gray planes denote the environment boundaries; the hollow rectangular region enclosed by these planes serves as the execution space for navigation tasks. The dark blue sphere represents the target point, light blue cubes represent obstacles, green rays denote LiDAR beams that do not detect obstacles, and red rays denote LiDAR beams that detect obstacles. The center of these LiDAR beams corresponds to the UAV.

Table 1

The parameter settings of the simulation environment.

parameters	description	unit	value
LENGTH	The length of the simulation environment	m	600
WIDTH	The width of the simulation environment	m	500
HEIGHT	The height of the simulation environment	m	500
INF_R	The longest detection distance of LiDAR	m	40
TAR_R	The radius of the spherical target point	m	30
MAX_V	The maximum velocity of the UAV	m/s	20
TIME_STEP	The time step of the simulation environment	s	0.02
MAX_STEP	The maximum time step of each episode	–	8000
DECISION_PERIOD	The number of time steps between decisions	–	5
OBS_NUM	The number of obstacles	–	configurable
OBS_V	The velocity of obstacles	m/s	configurable
OBS_L_MIN	The lower bound of obstacle side length	m	configurable
OBS_L_MAX	The upper bound of obstacle side length	m	configurable

4.2. Baselines

Because of SAC's advantages in environmental exploration for complex continuous control problems, we choose it as the baseline for navigation methods. Based on SAC, we compare HAP with two other reactive navigation methods and two ablation variants, as follows:

- 1) SAC: An entropy-maximizing DRL algorithm that simultaneously optimizes the expected return while maximizing policy entropy, yielding more robust policies. The input uses the same feature-extraction neural network as Planner (as shown in Fig. 8).
- 2) LSTM-SAC: A variant where SAC employs the same temporal neural network as Replanning Trigger (as shown in Fig. 9) for input feature extraction, enabling reactive navigation.

Table 2
Training hyperparameters of HAP.

Training hyperparameters	Planner	Replanning Trigger
Number of training episodes E	2×10^5	2×10^5
Discount Factor γ	0.99	0.99
Soft update factor τ	0.005	–
Learning rate	3×10^{-4}	1×10^{-4}
Batchsize	256	256
Exploration noise ϵ	–	0.05
Initial entropy	auto	–
Target entropy	auto	–
Reply buffer size	1×10^6	5×10^5
Backbone network	[512x256x256]	[256x256x128]
Reward function coefficient σ_1	–15	–
Reward function coefficient σ_2	0.05	–
Reward function coefficient σ_3	–8	–
Reward function coefficient σ_4	–0.5	–
Reward function coefficient σ_5	–	–0.1
Reward function coefficient σ_6	–	0.5
r_{arrive}	20	20
r_{crash}	–20	–20
$r_{timeout}$	–20	–20

- 3) Planner-only: A navigation method using only the DSAC-trained Planner, where replanning is triggered solely upon completion of path tracking. Since the action space in Eq. (9) enforces paths to extend to the maximum detection range (INF_R), this baseline evaluates the execution of long-term plans without intermediate updates.
- 4) HAP-SAC: An ablation variant of HAP in which the Planner is trained using SAC (instead of DSAC), designed to verify the effectiveness of DSAC.

To ensure a fair comparison against baselines, we adopt key DRL parameters from established literature on continuous and discrete control tasks [35]. The reward function coefficients, which are specific to our proposed method, are determined through a systematic pre-experimental sensitivity analysis. All the hyperparameters involved in the training phase are shown in Table 2. All algorithms are implemented and evaluated on a workstation equipped with an Intel Core i5-13600KF 14-core processor and an NVIDIA GeForce RTX 4070 GPU.

4.3. Training and results

To comprehensively assess the navigation performance of each method in task environments, we define quantitative evaluation metrics from aspects of task safety and execution efficiency, as shown in Table 3. These metrics are selected to distinctly evaluate task reliability and execution efficiency. Specifically, SR, CR, and LR quantify the safety and robustness of the navigation policy, while TIME measures the efficiency of valid executions. We calculate TIME exclusively for successful episodes to ensure statistical validity, as failed trajectories exhibit heterogeneous duration distributions ranging from early-stage collisions to maximum-step timeouts. Aggregating these disparate failure modes into the time metric would obscure the true efficiency of the model.

We train HAP and the other three methods under the same setting, i.e., OBS_NUM = 100, (OBS_L_MIN, OBS_L_MAX) = (20, 40), OBS_V = 0. To ensure statistical reliability and quantify the variance arising from initialization and training dynamics, we train each method using 10 independent random seeds. Consequently, all quantitative results are reported as the mean $\pm$ standard deviation across these 10 models, where each model is evaluated over 1000 test episodes. Their performance in the training environment is shown in Table 4.

Table 4 shows that HAP outperforms other methods, achieving the highest SR, closely followed by HAP-SAC. This result illustrates the advantage of DSAC in path generation for complex environments. The

Table 3
Quantitative evaluation indicators for navigation tasks.

Evaluation indicators	Descriptions
Success Rate (SR) (%)	The percentage of episodes in which the UAV successfully reaches the target.
Collision Rate (CR) (%)	The percentage of episodes in which the UAV collides with obstacles or environmental boundaries.
Lost Rate (LR) (%)	The percentage of episodes in which the UAV remains hovering in the environment until the episode ends.
TIME	The average number of time steps taken for episodes in which the UAV successfully reaches the target.

Table 4
Performance of different methods in the training environment. Results are reported as mean $\pm$ standard deviation across 10 independent random seeds.

Methods	Metrics			
	SR(%) $\uparrow$	CR(%) $\downarrow$	LR(%) $\downarrow$	TIME $\downarrow$
SAC	81.66 $\pm$ 1.38	17.97 $\pm$ 1.29	0.37 $\pm$ 0.19	1459.75 $\pm$ 8.98
LSTM-SAC	97.56 $\pm$ 0.83	2.13 $\pm$ 0.76	0.31 $\pm$ 0.20	1491.33 $\pm$ 9.18
Planner-only	96.73 $\pm$ 0.61	3.27 $\pm$ 0.61	0	1717.10 $\pm$ 19.94
HAP-SAC	97.62 $\pm$ 0.47	2.38 $\pm$ 0.47	0	1530.62 $\pm$ 10.25
HAP	98.62 $\pm$ 0.57	1.38 $\pm$ 0.57	0	1644.42 $\pm$ 11.85

performance of Planner-only is also lower than that of HAP. For reactive navigation methods, SAC, despite sharing the same feature extractor structure as Planner-only, performs significantly worse. Even with the integration of temporal information based on SAC, LSTM-SAC's SR still falls short of that of HAP. Notably, although reactive methods exhibit relatively shorter TIME, this metric should be interpreted alongside their lower SR and higher CR. This indicates a greedy behavioral pattern driven by inherent short-sightedness. Lacking the long-term vision of planning-based methods, reactive agents tend to attempt risky, straight-line paths that frequently result in collisions. In contrast, the TIME of planning-based methods is marginally higher because it prioritizes safety and trajectory smoothness. By generating Bézier curves to actively navigate around obstacles, HAP accepts a slight increase in flight time to secure a significantly higher SR and robust obstacle avoidance.

4.4. Environment generalization test

To compare the navigation performance and adaptability of each method in complex, dynamic environments, we freeze the parameters of each trained method and construct three distinct scenarios by adjusting the obstacle-related parameters to test their environmental generalization capability. The final performance metric for each method is the mean value of test results across 10 models, where each model was derived from an independent, separate random training session, and each was evaluated over 1000 test episodes.

4.4.1. Environment I: more obstacles

To evaluate the generalization performance of different models across varying obstacle counts, we set OBS_NUM to 120, 140, 160, 180, and 200, creating test scenarios with incrementally increasing obstacle numbers. Other obstacle parameters were fixed as OBS_V = 0 and (OBS_L_MIN, OBS_L_MAX) = (20, 40), consistent with the training phase. As the number of obstacles increases, the UAV's maneuverable space decreases, resulting in a gradual increase in navigation task difficulty.

As seen in Table 5, HAP consistently outperforms other methods, achieving the highest SR across all obstacle count tests. It is worth noting that while the performance gap between HAP and baselines is relatively narrow in simpler environments due to performance saturation, this gap widens significantly as task complexity increases. For instance, in

Table 5

Performance of different methods in environments with more obstacles. Results are reported as mean $\pm$ standard deviation across 10 independent random seeds.

OBS_NUM	Methods	Metrics			
		SR(%) $\uparrow$	CR(%) $\downarrow$	LR(%) $\downarrow$	TIME $\downarrow$
120	SAC	79.86 $\pm$ 1.37	19.79 $\pm$ 1.36	0.35 $\pm$ 0.17	1463.89 $\pm$ 9.60
	LSTM-SAC	97.08 $\pm$ 0.68	2.55 $\pm$ 0.64	0.37 $\pm$ 0.16	1501.38 $\pm$ 9.86
	Planner-only	94.52 $\pm$ 0.80	5.48 $\pm$ 0.80	0	1724.25 $\pm$ 15.33
	HAP-SAC	96.00 $\pm$ 0.48	4.00 $\pm$ 0.48	0	1570.73 $\pm$ 17.81
	HAP	97.29 $\pm$ 0.53	2.71 $\pm$ 0.53	0	1646.65 $\pm$ 10.95
140	SAC	76.14 $\pm$ 1.55	23.53 $\pm$ 1.53	0.33 $\pm$ 0.20	1472.03 $\pm$ 9.31
	LSTM-SAC	96.29 $\pm$ 0.56	3.24 $\pm$ 0.51	0.47 $\pm$ 0.24	1514.45 $\pm$ 11.31
	Planner-only	92.79 $\pm$ 0.69	7.21 $\pm$ 0.69	0	1740.79 $\pm$ 17.86
	HAP-SAC	95.15 $\pm$ 0.70	4.85 $\pm$ 0.70	0	1580.64 $\pm$ 12.25
	HAP	97.17 $\pm$ 0.62	2.83 $\pm$ 0.62	0	1646.88 $\pm$ 13.67
160	SAC	71.39 $\pm$ 1.43	28.19 $\pm$ 1.39	0.42 $\pm$ 0.15	1479.55 $\pm$ 9.50
	LSTM-SAC	94.12 $\pm$ 0.79	5.37 $\pm$ 0.76	0.51 $\pm$ 0.21	1529.74 $\pm$ 11.18
	Planner-only	91.25 $\pm$ 1.20	8.75 $\pm$ 1.20	0	1753.90 $\pm$ 16.13
	HAP-SAC	93.84 $\pm$ 0.56	6.16 $\pm$ 0.56	0	1612.49 $\pm$ 10.71
	HAP	96.18 $\pm$ 0.59	3.82 $\pm$ 0.59	0	1645.42 $\pm$ 13.63
180	SAC	67.29 $\pm$ 1.50	32.27 $\pm$ 1.58	0.44 $\pm$ 0.21	1484.82 $\pm$ 9.27
	LSTM-SAC	92.61 $\pm$ 0.92	6.46 $\pm$ 0.89	0.93 $\pm$ 0.18	1537.87 $\pm$ 15.62
	Planner-only	88.82 $\pm$ 1.07	11.18 $\pm$ 1.07	0	1767.78 $\pm$ 14.21
	HAP-SAC	92.35 $\pm$ 1.30	7.64 $\pm$ 1.32	0.01 $\pm$ 0.03	1616.61 $\pm$ 18.93
	HAP	95.63 $\pm$ 0.65	4.37 $\pm$ 0.65	0	1657.76 $\pm$ 14.72
200	SAC	65.03 $\pm$ 1.49	34.52 $\pm$ 1.48	0.45 $\pm$ 0.10	1487.70 $\pm$ 10.94
	LSTM-SAC	91.40 $\pm$ 0.97	6.25 $\pm$ 0.94	2.35 $\pm$ 0.18	1543.94 $\pm$ 14.59
	Planner-only	86.78 $\pm$ 0.78	13.22 $\pm$ 0.78	0	1767.89 $\pm$ 18.82
	HAP-SAC	90.11 $\pm$ 0.68	9.88 $\pm$ 0.66	0.01 $\pm$ 0.03	1667.72 $\pm$ 23.40
	HAP	95.33 $\pm$ 0.54	4.67 $\pm$ 0.54	0	1644.42 $\pm$ 13.92

the scenario with 200 obstacles, HAP maintains a high SR of 95.33% $\pm$ 0.54%, whereas the second-best method drops to 91.40% $\pm$ 0.97%. The non-overlapping performance intervals (mean $\pm$ standard deviation) in such challenging conditions provide strong statistical evidence that HAP's performance gains are statistically distinguishable from the baselines. Notably, while SAC maintains the shortest TIME, its SR remains far lower than that of other methods. This is because SAC only leverages current environmental information and lacks planning capability, resulting in its ability to only execute simple target-approaching behaviors rather than reasonably avoiding obstacles. In contrast, LSTM-SAC significantly improves SR by integrating temporal information into SAC's framework. However, its SR still lags behind HAP, and it exhibits weaker environment generalization, which is reflected in a more rapid decline in SR as the number of obstacles increases. When compared to SAC, Planner-only's SR is consistently much higher. This demonstrates the effectiveness of its action space, which is defined by the control points of planned Bézier curves. Nevertheless, Planner-only's SR remains lower than HAP's and decreases more sharply with increasing obstacles. This result highlights that Replanning Trigger enables the UAV to adaptively balance planning and real-time reaction, better accommodating complex navigation tasks. Finally, HAP-SAC's performance is marginally lower than that of HAP, confirming the advantage of DSAC in path planning.

Fig. 11 illustrates the typical navigation process of various methods for the same task under the configuration with the maximum obstacle count (OBS_NUM = 200). The reactive methods SAC and LSTM-SAC exhibit noticeable oscillatory behavior in their trajectories due to frequent maneuvering commands. This behavior is significantly more susceptible to collisions in dense obstacle environments. Specifically, SAC collides with an obstacle merely by attempting to fly over it, even when substantial maneuvering space remains. LSTM-SAC performs slightly better due to the incorporation of temporal information, but it still encounters the freezing robot problem in narrow gaps between obstacles, ultimately resulting in a collision. In contrast, HAP and its two ablation variants exhibit relatively smooth trajectories due to their long-term planning capability. However, in this task, Planner-only experiences a collision

due to reduced path reliability in the later stages, while HAP-SAC oscillates back and forth within dense obstacle clusters, ultimately colliding with an obstacle. Only HAP triggers replanning at the proper moment, generating a higher-quality path that ultimately reaches the target. This demonstrates that HAP, using a Planner trained via DSAC, can generate higher-quality paths, while the Replanning Trigger module achieves a dynamic balance between the planning-based and reactive behavior modes.

4.4.2. Environment II: larger obstacles

In this section, we aim to evaluate the generalization capabilities of different methods for larger obstacle sizes. We fix OBS_NUM = 100 and OBS_V = 0, which remain consistent with the training phase. Subsequently, the obstacle size ranges (OBS_L_MIN, OBS_L_MAX) are set to (25, 45), (30, 50), (35, 55), (40, 60), and (45, 65) to construct test scenarios with incrementally increasing obstacle sizes.

As obstacles increase in size, the UAV's maneuvering space becomes increasingly compressed, narrowing the feasible navigation paths. Furthermore, larger obstacles are more likely to create concave regions between adjacent obstacles, ultimately leading to a decrease in SR across all methods. Table 6 shows that HAP achieves the highest SR and best generalization performance (i.e., the smallest SR degradation) in all tests compared to other methods. Among the reactive methods, LSTM-SAC achieves a significant SR improvement over SAC—with SAC still performing the worst—but remains inferior to HAP. Notably, the SR of both reactive methods decreases significantly as obstacle size increases, along with a corresponding increase in LR. In contrast, HAP maintains an SR exceeding 80% even in environments with the largest obstacles ((OBS_L_MIN, OBS_L_MAX) = (45, 65)). Regarding the other two planning-based ablation methods, HAP-SAC's overall performance slightly trails HAP, while Planner-only further reduces reliability guarantees for the latter stages of generated paths due to occlusion by large obstacles, leading to a further decrease in SR.

As shown in Fig. 12, in the typical case with the largest obstacle size ((OBS_L_MIN, OBS_L_MAX) = (45, 65)), only HAP successfully

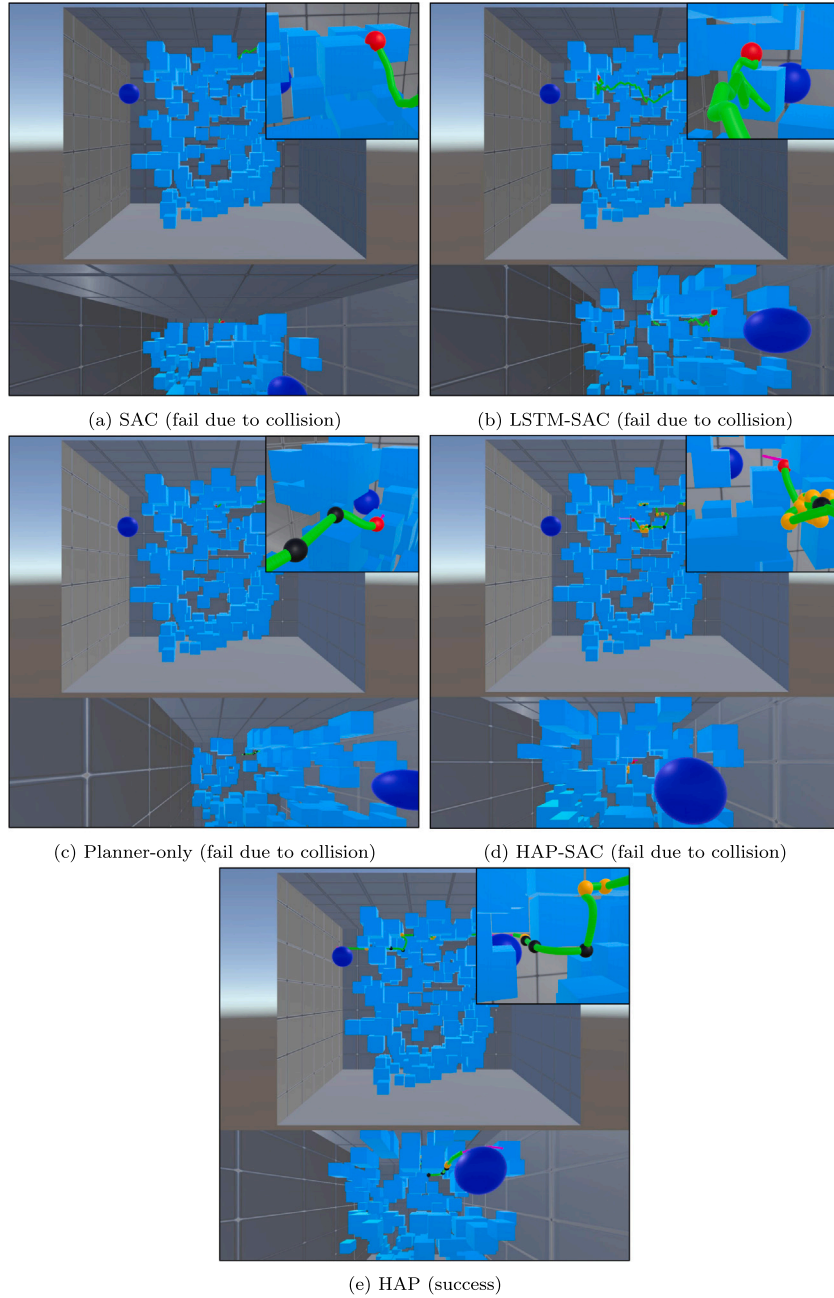


Fig. 11. Typical cases of the UAV using various methods to perform the same task when $OBS_NUM = 200$. The green solid lines represent trajectories of the UAV, black spheres denote replanning triggered at the end of current path, orange spheres denote replanning triggered by Replanning Trigger and red spheres denote collisions.

navigates the narrow gaps between obstacle clusters stably and ultimately reaches the target. In contrast, both SAC and LSTM-SAC collide with obstacles due to path oscillation issues. For Planner-only, it fails to traverse the gap because it possesses only long-term planning capabilities and lacks real-time obstacle avoidance. As for HAP-SAC, its Planner module (trained via SAC rather than DSAC) is insufficiently capable of generating reliable paths under such extreme conditions, leading to a collision.

4.4.3. Environment III: dynamic obstacles

Lastly, to evaluate the generalization capabilities of different models in more complex dynamic obstacle environments, we set OBS_V to 3, 6,

9, 12, and 15, and the obstacle collision coefficient to 1. Other obstacle-related parameters are kept fixed at $OBS_NUM = 100$ and $(OBS_L_MIN, OBS_L_MAX) = (20, 40)$ to align with the parameter settings of the training stage. The test results are shown in Table 7.

As shown in Table 7, SAC performs the worst in dynamic obstacle environments, while LSTM-SAC exhibits the best performance in terms of SR and TIME in low-speed obstacle scenarios ($OBS_V = 3$ and 6). This is because LSTM-SAC responds to obstacles in real time and captures their motion trends, enabling it to instantly steer the UAV away from collision danger when encountering obstacles. Notably, as obstacle speed continues to increase, LSTM-SAC's performance advantages in SR and TIME gradually diminish. In high-speed obstacle environments (OBS_V

Table 6

Performance of different methods in environments with larger obstacles. Results are reported as mean $\pm$ standard deviation across 10 independent random seeds.

(OBS_L_MIN, OBS_L_MAX)	Methods	Metrics			
		SR(%) $\uparrow$	CR(%) $\downarrow$	LR(%) $\downarrow$	TIME $\downarrow$
(25, 45)	SAC	74.79 $\pm$ 1.77	24.79 $\pm$ 1.63	0.42 $\pm$ 0.20	1457.75 $\pm$ 15.05
	LSTM-SAC	96.45 $\pm$ 0.64	2.83 $\pm$ 0.58	0.72 $\pm$ 0.29	1507.15 $\pm$ 15.72
	Planner-only	93.89 $\pm$ 0.56	6.11 $\pm$ 0.56	0	1749.44 $\pm$ 17.16
	HAP-SAC	94.70 $\pm$ 0.59	5.30 $\pm$ 0.59	0	1586.99 $\pm$ 12.76
	HAP	97.38 $\pm$ 0.58	2.62 $\pm$ 0.58	0	1648.26 $\pm$ 17.27
(30, 50)	SAC	66.72 $\pm$ 2.16	32.83 $\pm$ 2.09	0.45 $\pm$ 0.25	1458.22 $\pm$ 19.58
	LSTM-SAC	93.07 $\pm$ 0.98	5.94 $\pm$ 0.90	0.99 $\pm$ 0.31	1524.37 $\pm$ 15.19
	Planner-only	88.12 $\pm$ 1.11	11.88 $\pm$ 1.11	0	1756.43 $\pm$ 17.56
	HAP-SAC	92.34 $\pm$ 0.49	7.66 $\pm$ 0.49	0	1630.06 $\pm$ 22.34
	HAP	95.36 $\pm$ 0.54	4.64 $\pm$ 0.54	0	1648.88 $\pm$ 18.91
(35, 55)	SAC	55.57 $\pm$ 1.22	44.26 $\pm$ 1.24	0.17 $\pm$ 0.12	1515.15 $\pm$ 12.54
	LSTM-SAC	86.08 $\pm$ 0.85	12.40 $\pm$ 0.78	1.52 $\pm$ 0.21	1524.52 $\pm$ 17.92
	Planner-only	82.55 $\pm$ 0.72	17.45 $\pm$ 0.72	0	1792.61 $\pm$ 18.43
	HAP-SAC	90.54 $\pm$ 0.78	9.44 $\pm$ 0.80	0.02 $\pm$ 0.04	1690.41 $\pm$ 15.88
	HAP	93.81 $\pm$ 1.03	6.19 $\pm$ 1.03	0	1649.83 $\pm$ 18.26
(40, 60)	SAC	50.33 $\pm$ 2.36	49.44 $\pm$ 2.33	0.23 $\pm$ 0.17	1510.17 $\pm$ 11.03
	LSTM-SAC	80.66 $\pm$ 1.12	17.20 $\pm$ 1.10	2.14 $\pm$ 0.28	1525.19 $\pm$ 17.82
	Planner-only	76.48 $\pm$ 1.50	23.52 $\pm$ 1.50	0	1803.29 $\pm$ 21.20
	HAP-SAC	85.94 $\pm$ 0.83	14.00 $\pm$ 0.82	0.06 $\pm$ 0.08	1777.06 $\pm$ 18.72
	HAP	88.21 $\pm$ 0.71	11.79 $\pm$ 0.71	0	1695.62 $\pm$ 20.03
(45, 65)	SAC	48.09 $\pm$ 1.42	51.67 $\pm$ 1.41	0.24 $\pm$ 0.13	1486.34 $\pm$ 13.97
	LSTM-SAC	72.48 $\pm$ 1.04	24.40 $\pm$ 0.98	3.12 $\pm$ 0.34	1518.21 $\pm$ 21.62
	Planner-only	69.46 $\pm$ 1.60	30.54 $\pm$ 1.60	0	1815.50 $\pm$ 15.57
	HAP-SAC	77.68 $\pm$ 1.03	22.08 $\pm$ 1.05	0.24 $\pm$ 0.14	1777.28 $\pm$ 19.52
	HAP	83.23 $\pm$ 1.12	16.75 $\pm$ 1.17	0.02 $\pm$ 0.06	1717.38 $\pm$ 20.61

= 12 and 15), its performance is fully overtaken by HAP. For planning-based methods, HAP-SAC consistently performs marginally worse than HAP, further validating the effectiveness of DSAC. In contrast, Planner-only performs significantly worse than HAP, with its SR being 21.37% lower than that of HAP when OBS_V = 15. This indicates that navigation methods relying solely on planning face difficulties adapting to dynamic obstacle environments, primarily due to diminished path reliability in the later stages of navigation.

4.5. The effectiveness of replanning trigger

To further verify the adaptive capability of Replanning Trigger proposed in this paper, we compare its adaptive replanning mechanism with three typical rule-based replanning strategies [36], testing them in the three most challenging task environments described in Section 4.4:

- 1) Time-based: This strategy triggers replanning at fixed time intervals. A replan is executed when the number of elapsed time steps since the last replanning event reaches or exceeds a predefined threshold Δt .
- 2) Distance-based: This strategy triggers replanning based on traveled distance. A replan occurs when the cumulative distance traveled since the last replan is greater than or equal to a predefined fixed threshold Δd (m).
- 3) Stuck-based: This strategy triggers replanning by detecting UAV stagnant. Specifically, it is configured to trigger replanning when the UAV remains stagnant for 10 consecutive time steps.

We set the threshold values of Δt to 10, 20, and 30, and the threshold values of Δd (m) to 10, 20, and 30, respectively. These values cover replanning triggering patterns ranging from frequent to infrequent. The test results are shown in Table 8.

As shown in Table 8, in the three most difficult task environments, the Time-based strategy has the worst overall performance. This is because when TIME_STEP is set to a small value (0.02s) in this paper, it triggers replanning too frequently, resulting in a significant decrease in

SR and an increase in TIME. In contrast, the Distance-based strategy performs relatively well, especially when $\Delta d = 20$, because the UAV almost always triggers replanning halfway along the path. This prevents frequent replanning that can lead to track oscillations and, to a certain extent, avoids the problem of insufficient credibility of the latter path. However, even with this, the performance of the rule-based Distance-based strategy is still far inferior to that of HAP. As for the Stuck-based strategy, its performance was comparable to that of the Distance-based strategy in the first two task environments, but its SR was the lowest in the third. This is because the UAV is less likely to become stuck when faced with high-velocity obstacles. Instead, it requires timely replanning to cope with dynamic obstacles. In this case, the Stuck-based strategy is almost ineffective and may even increase TIME due to ineffective replanning.

As shown in Table 8, across the three most challenging task environments, the Time-based strategy exhibits the worst overall performance. This is because TIME_STEP in this study is set to a small value (0.02s), which causes the strategy to trigger replanning excessively frequently, leading to a significant drop in SR and a prolongation of TIME. In contrast, the Distance-based strategy performs relatively well, particularly when $\Delta d = 20$: a replan is almost always triggered halfway along the planned path. This avoids the frequent replanning that would otherwise cause trajectory oscillations and, to a certain extent, mitigates the issue of low reliability in later-stage paths. However, even with this advantage, the rule-based Distance-based strategy still performs far worse than HAP. Regarding the Stuck-based strategy, its performance is comparable to that of the Distance-based strategy in the first two task environments, but its SR is the lowest in the third. This is because the UAV is less likely to become stagnant when confronting high-velocity obstacles. Instead, the UAV requires timely replanning to adapt to dynamic obstacles. In summary, HAP demonstrates the strongest overall performance compared to the typical rule-based replanning strategies, evidenced by its highest SR and relatively short TIME. This further confirms that HAP's Replanning Trigger module enables real-time environmental observation, adaptive replanning triggering, and reliable, high-quality completion of navigation tasks.

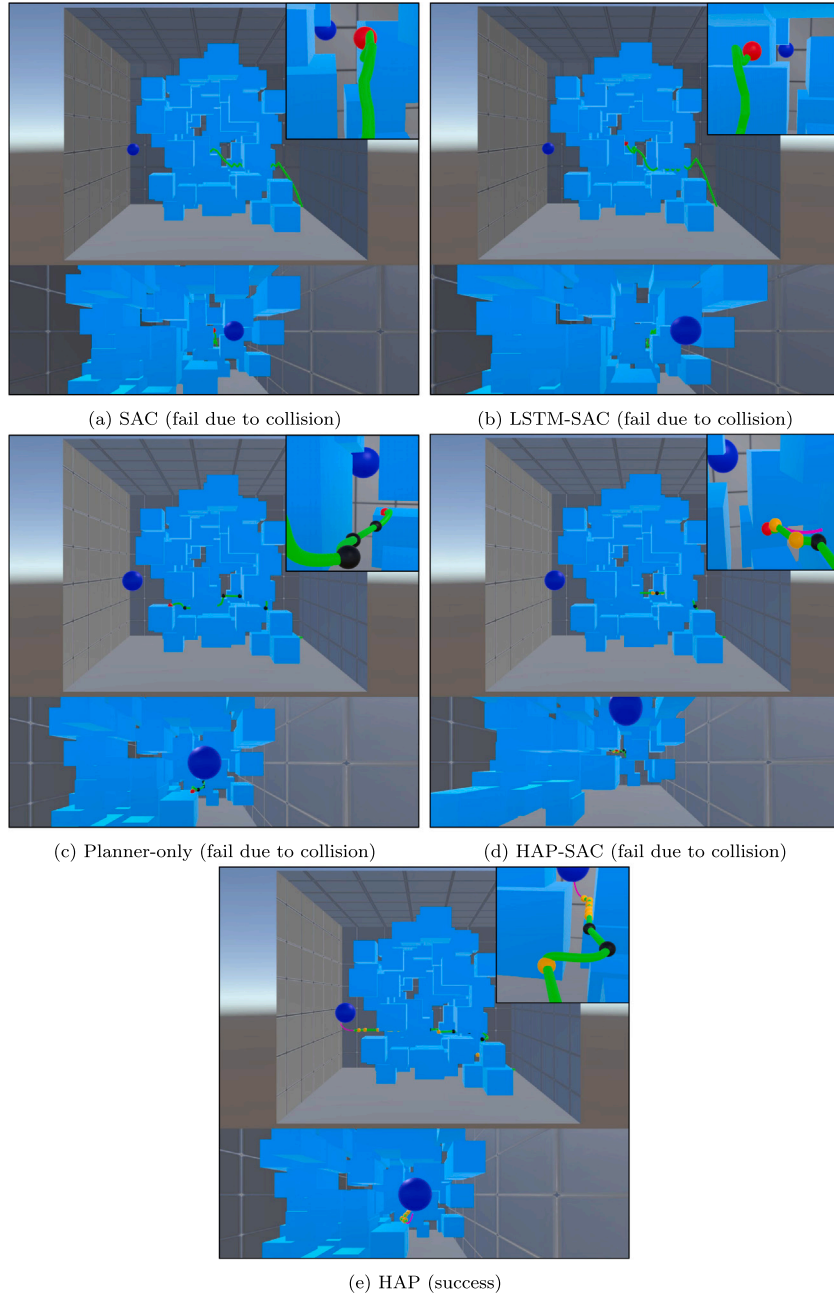


Fig. 12. Typical cases of the UAV using various methods to perform the same task when $(OBS_L_MIN, OBS_L_MAX) = (45, 65)$. The green solid lines represent trajectories of the UAV, black spheres denote replanning triggered at the end of current path, orange spheres denote replanning triggered by Replanning Trigger and red spheres denote collisions.

4.6. Sensitivity analysis of reward function

For Planner, the obstacle avoidance penalty $r_{avoidance}$ (Eq. 16) is paramount. An appropriate setting of σ_3 facilitates early convergence by enabling the agent to survive effectively and collect experiences during the later stages of navigation. However, an excessive penalty induces overly conservative behaviors that compromise task efficiency. Regarding the Replanning Trigger module, the core objective centers on optimizing the decision-making frequency, which is directly governed by the coefficient σ_5 within the replanning cost function $r_{progress}$ (Eq. 21). Consequently, we prioritize the tuning of σ_3 and σ_5 , while other coefficients primarily serve to scale reward signals to appropriate magnitudes. To verify the rationality of our configuration, we conduct sensitivity analyses on these two decisive parameters.

Specifically, we evaluate the model in the training environment by varying σ_3 and σ_5 relative to our configuration ($\sigma_3 = -8, \sigma_5 = -0.1$). The results are presented in Table 9.

As shown in Panel A, σ_3 balances safety and efficiency. Reducing the penalty magnitude (e.g., $\sigma_3 = -1$) weakens the repulsive force from obstacles. While this slight reduction in safety margins keeps TIME relatively low, it results in a higher CR, leading to a drop in the overall SR. On the other hand, excessive penalties (e.g., $\sigma_3 = -16$) induce conservative behaviors. The UAV tends to take longer detours to maintain large safety margins, which is reflected in the significantly increased TIME. Panel B demonstrates the trade-off between responsiveness and stability. With a negligible cost (e.g., $\sigma_5 = -0.01$), Replanning Trigger activates Planner frequently. Although this results in the shortest TIME

Table 7

Performance of different methods in environments with dynamic obstacles. Results are reported as mean $\pm$ standard deviation across 10 independent random seeds.

OBS_V	Methods	Metrics			
		SR(%) $\uparrow$	CR(%) $\downarrow$	LR(%) $\downarrow$	TIME $\downarrow$
3	SAC	77.01 $\pm$ 1.40	22.98 $\pm$ 1.40	0.01 $\pm$ 0.03	1522.24 $\pm$ 11.88
	LSTM-SAC	96.71 $\pm$ 1.23	3.27 $\pm$ 1.22	0.02 $\pm$ 0.04	1503.72 $\pm$ 19.05
	Planner-only	87.76 $\pm$ 1.01	12.24 $\pm$ 1.01	0	1718.17 $\pm$ 17.24
	HAP-SAC	93.20 $\pm$ 1.52	6.80 $\pm$ 1.52	0	1555.28 $\pm$ 25.94
	HAP	95.34 $\pm$ 1.39	4.66 $\pm$ 1.39	0	1633.54 $\pm$ 27.26
6	SAC	72.77 $\pm$ 0.69	27.21 $\pm$ 0.69	0.02 $\pm$ 0.04	1598.58 $\pm$ 17.27
	LSTM-SAC	93.85 $\pm$ 1.38	6.15 $\pm$ 1.38	0	1575.07 $\pm$ 19.78
	Planner-only	77.00 $\pm$ 0.82	23.00 $\pm$ 0.82	0	1708.98 $\pm$ 18.11
	HAP-SAC	84.55 $\pm$ 1.49	15.45 $\pm$ 1.49	0	1557.67 $\pm$ 22.73
	HAP	88.07 $\pm$ 1.47	11.93 $\pm$ 1.47	0	1629.32 $\pm$ 36.18
9	SAC	68.20 $\pm$ 1.14	31.79 $\pm$ 1.16	0.01 $\pm$ 0.03	1737.87 $\pm$ 29.54
	LSTM-SAC	87.36 $\pm$ 1.21	12.62 $\pm$ 1.21	0.02 $\pm$ 0.04	1746.72 $\pm$ 30.61
	Planner-only	67.83 $\pm$ 0.98	32.17 $\pm$ 0.98	0	1697.03 $\pm$ 12.82
	HAP-SAC	82.68 $\pm$ 1.71	17.32 $\pm$ 1.71	0	1558.97 $\pm$ 18.26
	HAP	85.06 $\pm$ 1.45	14.94 $\pm$ 1.45	0	1669.74 $\pm$ 39.93
12	SAC	52.25 $\pm$ 3.38	47.65 $\pm$ 3.39	0.10 $\pm$ 0.13	2045.34 $\pm$ 80.82
	LSTM-SAC	75.02 $\pm$ 1.97	24.10 $\pm$ 1.95	0.88 $\pm$ 0.21	1813.28 $\pm$ 54.80
	Planner-only	61.23 $\pm$ 2.13	38.77 $\pm$ 2.13	0	1699.26 $\pm$ 17.09
	HAP-SAC	79.87 $\pm$ 1.28	20.13 $\pm$ 1.28	0	1561.85 $\pm$ 23.49
	HAP	81.21 $\pm$ 1.52	18.79 $\pm$ 1.52	0	1663.48 $\pm$ 32.25
15	SAC	36.35 $\pm$ 1.96	62.71 $\pm$ 1.88	0.94 $\pm$ 0.40	2237.07 $\pm$ 98.99
	LSTM-SAC	61.81 $\pm$ 1.88	36.27 $\pm$ 1.83	1.92 $\pm$ 0.22	2274.56 $\pm$ 73.91
	Planner-only	56.13 $\pm$ 1.47	43.87 $\pm$ 1.47	0	1693.85 $\pm$ 22.22
	HAP-SAC	75.21 $\pm$ 1.63	24.79 $\pm$ 1.63	0	1562.77 $\pm$ 24.50
	HAP	77.51 $\pm$ 1.25	22.49 $\pm$ 1.25	0	1656.55 $\pm$ 25.57

Table 8

Comparison of performance by different replanning strategies. Results are reported as mean $\pm$ standard deviation across 10 independent random seeds.

OBS_NUM	(OBS_L_MIN, OBS_L_MAX)	OBS_V	Methods		Metrics						
					SR(%) $\uparrow$	CR(%) $\downarrow$	LR(%) $\downarrow$	TIME $\downarrow$			
200	(20, 40)	0	Time-based	$\Delta t = 10$	62.14 $\pm$ 1.62	37.65 $\pm$ 1.69	0.21 $\pm$ 0.13	1849.11 $\pm$ 20.84			
				$\Delta t = 20$	65.85 $\pm$ 1.51	33.77 $\pm$ 1.54	0.38 $\pm$ 0.21	1907.35 $\pm$ 24.61			
				$\Delta t = 30$	70.75 $\pm$ 1.38	28.94 $\pm$ 1.33	0.31 $\pm$ 0.19	1787.42 $\pm$ 20.71			
			Distance-based	$\Delta d = 10$	71.89 $\pm$ 0.96	28.00 $\pm$ 0.93	0.11 $\pm$ 0.15	1766.73 $\pm$ 24.63			
				$\Delta d = 20$	86.76 $\pm$ 1.13	13.18 $\pm$ 1.09	0.06 $\pm$ 0.08	1644.71 $\pm$ 18.97			
				$\Delta d = 30$	85.38 $\pm$ 0.91	14.62 $\pm$ 0.91	0	1651.31 $\pm$ 15.72			
			Stuck-based		86.81 $\pm$ 0.87	13.19 $\pm$ 0.87	0	1788.08 $\pm$ 17.68			
				HAP	95.33 $\pm$ 0.54	4.67 $\pm$ 0.54	0	1644.42 $\pm$ 13.92			
			100	(45, 65)	0	Time-based	$\Delta t = 10$	50.43 $\pm$ 1.65	45.94 $\pm$ 1.60	3.63 $\pm$ 0.31	1984.93 $\pm$ 25.71
							$\Delta t = 20$	52.08 $\pm$ 1.58	44.17 $\pm$ 1.54	3.75 $\pm$ 0.33	1894.70 $\pm$ 27.61
$\Delta t = 30$	57.98 $\pm$ 1.56	39.17 $\pm$ 1.58					2.85 $\pm$ 0.25	1916.79 $\pm$ 28.47			
Distance-based	$\Delta d = 10$	59.41 $\pm$ 1.44				37.88 $\pm$ 1.46	2.71 $\pm$ 0.18	1927.57 $\pm$ 18.29			
	$\Delta d = 20$	74.13 $\pm$ 1.23				25.60 $\pm$ 1.21	0.27 $\pm$ 0.12	1774.76 $\pm$ 18.02			
	$\Delta d = 30$	69.08 $\pm$ 1.34				30.92 $\pm$ 1.34	0	1691.90 $\pm$ 17.39			
Stuck-based		70.28 $\pm$ 1.29				29.72 $\pm$ 1.29	0	1839.50 $\pm$ 20.37			
	HAP	83.23 $\pm$ 1.12				16.75 $\pm$ 1.17	0.02 $\pm$ 0.06	1717.38 $\pm$ 20.61			
100	(20, 40)	15				Time-based	$\Delta t = 10$	59.87 $\pm$ 1.99	40.13 $\pm$ 1.99	0	1590.82 $\pm$ 18.62
							$\Delta t = 20$	59.85 $\pm$ 1.45	40.15 $\pm$ 1.45	0	1524.95 $\pm$ 20.74
			$\Delta t = 30$	58.48 $\pm$ 1.68	41.52 $\pm$ 1.68		0	1488.63 $\pm$ 17.76			
			Distance-based	$\Delta d = 10$	61.76 $\pm$ 1.23	38.24 $\pm$ 1.23	0	1501.94 $\pm$ 23.61			
				$\Delta d = 20$	67.86 $\pm$ 1.46	32.14 $\pm$ 1.46	0	1478.83 $\pm$ 21.21			
				$\Delta d = 30$	66.77 $\pm$ 1.29	33.23 $\pm$ 1.29	0	1527.99 $\pm$ 19.04			
			Stuck-based		57.86 $\pm$ 1.46	42.14 $\pm$ 1.46	0	1789.45 $\pm$ 28.13			
				HAP	77.51 $\pm$ 1.25	22.49 $\pm$ 1.25	0	1656.55 $\pm$ 25.57			

by continuously seeking local shortcuts, it compromises flight stability, leading to a higher CR. Conversely, a high cost (e.g., $\sigma_5 = -1.0$) discourages Replanning Trigger unless under extreme threat. This lazy behavior delays the response to dynamic obstacles, also increasing the

CR. Overall, these results indicate that our method exhibits robustness to parameter variations within a reasonable range. Our selected configuration represents a rational choice that effectively balances safety, efficiency, and stability.

Table 9

Sensitivity analysis of critical reward hyperparameters. Results are reported as mean $\pm$ standard deviation across 10 independent random seeds.

Reward Settings	Metrics			
	SR(%) $\uparrow$	CR(%) $\downarrow$	LR(%) $\downarrow$	TIME $\downarrow$
Panel A: Impact of Obstacle Avoidance Penalty (σ_3)				
$\sigma_3 = -1$	97.52 $\pm$ 0.51	2.48 $\pm$ 0.51	0	1643.68 $\pm$ 11.77
$\sigma_3 = -4$	98.06 $\pm$ 0.49	1.94 $\pm$ 0.49	0	1640.78 $\pm$ 12.47
$\sigma_3 = -8$ (Ours)	98.62 $\pm$ 0.57	1.38 $\pm$ 0.57	0	1644.42 $\pm$ 11.85
$\sigma_3 = -12$	97.75 $\pm$ 0.59	2.25 $\pm$ 0.59	0	1662.68 $\pm$ 18.64
$\sigma_3 = -16$	97.37 $\pm$ 0.58	2.63 $\pm$ 0.58	0	1678.26 $\pm$ 14.98
Panel B: Impact of Replanning Cost (σ_5)				
$\sigma_5 = -0.01$	97.21 $\pm$ 0.52	2.79 $\pm$ 0.52	0	1622.88 $\pm$ 31.21
$\sigma_5 = -0.05$	97.42 $\pm$ 0.41	2.58 $\pm$ 0.41	0	1630.56 $\pm$ 13.54
$\sigma_5 = -0.1$ (Ours)	98.62 $\pm$ 0.57	1.38 $\pm$ 0.57	0	1644.42 $\pm$ 11.85
$\sigma_5 = -0.5$	98.03 $\pm$ 0.42	1.97 $\pm$ 0.42	0	1640.86 $\pm$ 11.19
$\sigma_5 = -1.0$	97.62 $\pm$ 0.51	2.38 $\pm$ 0.51	0	1645.40 $\pm$ 27.56

5. Conclusion

In this paper, we propose a Hierarchical Adaptive Navigation Planner (HAP) for UAV navigation in complex 3D environments, which is based on deep reinforcement learning (DRL) and comprises two core modules: Planner and Replanning Trigger. Experimental training and validation were conducted in Unity-based simulation environments, and the results confirm the following conclusions. First, HAP demonstrates superior overall performance compared to DRL-based reactive navigation methods across multiple evaluation metrics, highlighting its comprehensive advantages in navigation performance. Second, the effectiveness of HAP's core modules is fully validated. For Planner, it can reduce action space dimensionality and generate high-quality paths via a customized Bézier curve-based control point selection mechanism. Besides, the state-of-the-art DSAC algorithm allows Planner to achieve a higher performance ceiling. As for Replanning Trigger, which leverages three consecutive frames of environmental information for adaptive replanning, it effectively balances planning stability and real-time obstacle avoidance. Third, the dense reward functions designed for both modules reliably ensure model convergence. In summary, HAP provides a robust solution for UAV navigation in complex 3D scenarios.

While HAP has demonstrated effective performance, it currently lacks sufficient deployment and validation in real-world scenarios. A notable sim-to-real discrepancy persists, as existing simulation frameworks fail to fully encapsulate the breadth of real-world challenges. For instance, real-world sensors are inherently prone to noise and latency, while idealized physical models overlook actuator delays and aerodynamic perturbations such as wind disturbances. Future work will focus on a multi-stage validation strategy, starting with Hardware-in-the-Loop (HIL) simulation for computational validation, followed by progressively complex real-world experiments from controlled indoor to outdoor environments. Moreover, domain randomization techniques will be investigated to systematically enhance the model's robustness for practical applications.

CRedit authorship contribution statement

Shuyuan Liu: Writing – review & editing, Writing – original draft, Visualization, Validation, Project administration, Methodology, Investigation. **Yu Zhang:** Writing – review & editing, Methodology, Formal analysis. **Guanxiong Li:** Writing – review & editing, Funding acquisition. **Hao Chen:** Supervision, Formal analysis, Data curation. **Xiaogang Deng:** Supervision, Resources.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work, the authors used DeepSeek-R1 to refine the language. After using this tool, the authors reviewed and

edited the content as needed and take full responsibility for the content of the published article.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This research was partially supported by the National Natural Science Foundation of China (grant number 12202297), the Open Funding of the National Key Laboratory of Fundamental Algorithms and Models for Engineering Simulation, the Open Funding of the National Key Laboratory of Digital and Agile Aircraft Design, and the Sichuan University Interdisciplinary Innovation Fund.

Data availability

Data will be made available upon request.

References

- [1] M.R. Rezaee, N.A.W.A. Hamid, M. Hussin, Z.A. Zukarnain, Comprehensive review of drones collision avoidance schemes: challenges and open issues, *IEEE Trans. Intell. Transp. Syst.* 25 (7) (2024) 6397–6426, <https://doi.org/10.1109/TITS.2024.3375893>
- [2] Y. Zhou, X. Kong, K.-P. Lin, L. Liu, Novel task decomposed multi-agent twin delayed deep deterministic policy gradient algorithm for multi-uav autonomous path planning, *Knowl.-Based Syst.* 287 (2024) 111462, <https://doi.org/10.1016/j.knsys.2024.111462>, <https://www.sciencedirect.com/science/article/pii/S0950705124000972>
- [3] J. Wang, B. Zeng, L. Deng, Z. Ji, C. Wei, Z. Zeng, Cognitive UAV tracking: leveraging DRL and hybrid curriculum learning for target reacquisition, *IEEE Trans. Autom. Sci. Eng.* 22 (2025) 16547–16559, <https://doi.org/10.1109/TASE.2025.3577984>
- [4] A. Ayala, L. Portela, F. Buarque, B.J. Fernandes, F. Cruz, UAV control in autonomous object-goal navigation: a systematic literature review, *Artif. Intell. Rev.* 57 (5) (2024) 125, <https://doi.org/10.1007/s10462-024-10758-7>
- [5] G. Sonugür, A review of quadrotor UAV: control and slam methodologies ranging from conventional to innovative approaches, *Robot. Auton. Syst.* 161 (2023) 104342, <https://doi.org/10.1016/j.robot.2022.104342>
- [6] K. Chen, J. Xiao, J. Liu, Q. Tong, H. Zhang, R. Liu, J. Zhang, A. Ajoudani, S. Chen, Semantic visual simultaneous localization and mapping: a survey, *IEEE Trans. Intell. Transp. Syst.* 26 (6) (2025) 7426–7449, <https://doi.org/10.1109/TITS.2025.3556928>
- [7] Y. Zhang, W. Zhao, J. Wang, Y. Yuan, Recent progress, challenges and future prospects of applied deep reinforcement learning: a practical perspective in path planning, *Neurocomputing* 608 (2024) 128423, <https://doi.org/10.1016/j.neucom.2024.128423>, <https://www.sciencedirect.com/science/article/pii/S09525231224011949>
- [8] Y. Xue, W. Chen, Multi-agent deep reinforcement learning for UAVS navigation in unknown complex environment, *IEEE Trans. Intell. Veh.* 9 (1) (2024) 2290–2303, <https://doi.org/10.1109/ITV.2023.3298292>
- [9] J.H. Chan, K. Liu, Y. Chen, A.S. Sagar, Y.-G. Kim, Reinforcement learning-based drone simulators: survey, practice, and challenge, *Artif. Intell. Rev.* 57 (10) (2024) 281.
- [10] J. Duan, W. Wang, L. Xiao, J. Gao, S.E. Li, C. Liu, Y.-Q. Zhang, B. Cheng, K. Li, Distributional soft actor-critic with three refinements, *IEEE Trans. Pattern Anal. Mach. Intell.* 47 (5) (2025) 3935–3946, <https://doi.org/10.1109/TPAMI.2025.3537087>
- [11] V. Mnih, K. Kavukcuoglu, D. Silver, A.A. Rusu, J. Veness, M.G. Bellemare, A. Graves, M. Riedmiller, A.K. Fiedelnd, G. Ostrovski, et al., Human-level control through deep reinforcement learning, *Nature* 518 (7540) (2015) 529–533, <https://doi.org/10.1038/nature14236>
- [12] Y. Ma, A. Cramariuc, F. Farshidian, M. Hutter, Learning coordinated badminton skills for legged manipulators, *Sci. Robot.* 10 (102) (2025) eadu3922, <https://doi.org/10.1126/scirobotics.adu3922>
- [13] J. Leng, C. Sun, H. Dong, D. Li, C. Zhang, P.C.Y. Chen, Adaptive eco-driving with guided speed planning and lane changing through signalized intersections, *IEEE Trans. Transp. Electrification* 11 (3) (2025) 8365–8376, <https://doi.org/10.1109/TTE.2025.3540057>
- [14] E. Kaufmann, L. Bauersfeld, A. Loquercio, M. Müller, V. Koltun, D. Scaramuzza, Champion-level drone racing using deep reinforcement learning, *Nature* 620 (7976) (2023) 982–987, <https://doi.org/10.1038/s41586-023-06419-4>
- [15] J. Zhang, J.T. Springenberg, J. Boedecker, W. Burgard, Deep reinforcement learning with successor features for navigation across similar environments, in: 2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), IEEE, 2017, pp. 2371–2378, <https://doi.org/10.1109/IROS.2017.8206049>
- [16] H. Huang, Y. Yang, H. Wang, Z. Ding, H. Sari, F. Adachi, Deep reinforcement learning for UAV navigation through massive MIMO technique, *IEEE Trans. Veh. Technol.* 69 (1) (2020) 1117–1121, <https://doi.org/10.1109/TVT.2019.2952549>

- [17] E. Marchesini, A. Farinelli, Discrete deep reinforcement learning for mapless navigation, in: 2020 IEEE International Conference on Robotics and Automation (ICRA), 2020, pp. 10688–10694, <https://doi.org/10.1109/ICRA40945.2020.9196739>
- [18] C. Wang, J. Wang, J. Wang, X. Zhang, Deep-reinforcement-learning-based autonomous UAV navigation with sparse rewards, *IEEE Internet Things J.* 7 (7) (2020) 6180–6190, <https://doi.org/10.1109/JIOT.2020.2973193>
- [19] Z. Liu, Y. Cao, J. Chen, J. Li, A hierarchical reinforcement learning algorithm based on attention mechanism for UAV autonomous navigation, *IEEE Trans. Intell. Transp. Syst.* 24 (11) (2023) 13309–13320, <https://doi.org/10.1109/TITS.2022.3225721>
- [20] S. Zhang, Y. Li, Q. Dong, Autonomous navigation of UAV in multi-obstacle environments based on a deep reinforcement learning approach, *Appl. Soft Comput.* 115 (2022) 108194, <https://doi.org/10.1016/j.asoc.2021.108194>
- [21] X. Li, J. Fang, K. Du, K. Mei, J. Xue, UAV obstacle avoidance by human-in-the-loop reinforcement in arbitrary 3d environment, in: 2023 42nd Chinese Control Conference (CCC), 2023, pp. 3589–3595, <https://doi.org/10.23919/CCC58697.2023.10240962>
- [22] L. Zhang, J. Peng, W. Yi, H. Lin, L. Lei, X. Song, A state-decomposition DDPG algorithm for UAV autonomous navigation in 3-d complex environments, *IEEE Internet Things J.* 11 (6) (2024) 10778–10790, <https://doi.org/10.1109/JIOT.2023.3327753>
- [23] Y. Xue, W. Chen, A UAV navigation approach based on deep reinforcement learning in large cluttered 3d environments, *IEEE Trans. Veh. Technol.* 72 (3) (2023) 3001–3014, <https://doi.org/10.1109/TVT.2022.3218855>
- [24] Y. Xue, W. Chen, Rloplanner: combining learning and motion planner for UAV safe navigation in cluttered unknown environments, *IEEE Trans. Veh. Technol.* 73 (4) (2024) 4904–4917, <https://doi.org/10.1109/TVT.2023.3330703>
- [25] Y. Xue, W. Chen, Combining motion planner and deep reinforcement learning for UAV navigation in unknown environment, *IEEE Robot. Autom. Lett.* 9 (1) (2024) 635–642, <https://doi.org/10.1109/LRA.2023.3334978>
- [26] W. Chen, Y. Xue, Hierarchical UAV autonomous navigation algorithm based on event-triggered deep reinforcement learning, *IEEE Trans. Veh. Technol.* (2025) 1–12, <https://doi.org/10.1109/TVT.2025.3584964>
- [27] Q. Luo, Y. Li, J. Zeng, G. Wu, Y. Wang, Quadrotor navigation considering attitude: a deep reinforcement learning method using tangent path rewards, *Expert Syst. Appl.* 298 (2026) 129762, <https://doi.org/10.1016/j.eswa.2025.129762>, <https://www.sciencedirect.com/science/article/pii/S0957417425033779>
- [28] J. Lee, A. Agarwal, C. Dann, T. Zhang, Learning in pomdps is sample-efficient with hindsight observability, in: International Conference on Machine Learning, PMLR, 2023, pp. 18733–18773.
- [29] M. Uehara, A. Sekhari, J.D. Lee, N. Kallus, W. Sun, Provably efficient reinforcement learning in partially observable dynamical systems, in: S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, A. Oh (Eds.), *Advances in Neural Information Processing Systems*, vol. 35, Curran Associates, Inc., 2022, pp. 578–592.
- [30] L. Sharma, N. Buono, A. Flather, X. Cai, J.P. How, Look before you LEAP: socially acceptable high-speed ground robot navigation in crowded hallways, in: 2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2024, pp. 3254–3259, <https://doi.org/10.1109/IROS58592.2024.10801967>
- [31] C.-L. Cheng, C.-C. Hsu, S. Saeedvand, J.-H. Jo, Multi-objective crowd-aware robot navigation system using deep reinforcement learning, *Appl. Soft Comput.* 151 (2024) 111154, <https://doi.org/10.1016/j.asoc.2023.111154>
- [32] W. Yu, J. Peng, Q. Qiu, H. Wang, L. Zhang, J. Ji, Pathrl: an end-to-end path generation method for collision avoidance via deep reinforcement learning, in: 2024 IEEE International Conference on Robotics and Automation (ICRA), IEEE, 2024, pp. 9278–9284.
- [33] F. AlMahamid, K. Grolinger, Autonomous unmanned aerial vehicle navigation using reinforcement learning: a systematic review, *Eng. Appl. Artif. Intell.* 115 (2022) 105321.
- [34] S. Pateria, B. Subagdja, A.-H. Tan, C. Quek, Hierarchical reinforcement learning: a comprehensive survey, *ACM Comput. Surv.* 54 (5) (Jun 2021), <https://doi.org/10.1145/3453160>
- [35] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, N. Dormann, Stable-baselines3: reliable reinforcement learning implementations, *J. Mach. Learn. Res.* 22 (268) (2021) 1–8, <http://jmlr.org/papers/v22/20-1364.html>
- [36] S. Macenski, T. Moore, D.V. Lu, A. Merzlyakov, M. Ferguson, From the desks of ros maintainers: a survey of modern & capable mobile Robotics algorithms in the robot operating system 2, *Robot. Auton. Syst.* 168 (2023) 104493.